

LECTURE 5

EC-241 OOP

© 2000 Prentice Hall, Inc. All rights reserved.



1

7.4 friend Functions and friend Classes

- **friend** function and **friend** classes
 - Can access **private** and **protected** members of another class
 - **friend** functions are not member functions of class
 - Defined outside of class scope
- Properties of friendship
 - Friendship is granted, not taken
 - Not symmetric (if **B** a **friend** of **A**, **A** not necessarily a **friend** of **B**)
 - Not transitive (if **A** a **friend** of **B**, **B** a **friend** of **C**, **A** not necessarily a **friend** of **C**)

© 2000 Prentice Hall, Inc. All rights reserved.



2

7.4 friend Functions and friend Classes

• friend declarations

- To declare a **friend** function
 - Type **friend** before the function prototype in the class that is giving friendship


```
friend int myFunction( int x );
```

 should appear in the class giving friendship
- To declare a **friend** class
 - Type **friend class Classname** in the class that is giving friendship
- if **ClassOne** is granting friendship to **ClassTwo**,


```
friend class ClassTwo;
```

 - should appear in **ClassOne**'s definition



```

1 // Fig. 7.6: fig07_06.cpp
2 // Non-friend/non-member functions cannot access
3 // private data of a class.
4 #include <iostream>
5
6 using std::cout;
7 using std::endl;
8
9 // Modified Count class
10 class Count {
11 public:
12     Count() { x = 0; } // constructor
13     void print() const { cout << x << endl; } // output
14 private:
15     int x; // data member
16 };
17
18 // Function tries to modify private data of Count,
19 // but cannot because it is not a friend of Count.
20 void cannotSetX( Count &c, int val )
21 {
22     c.x = val; // ERROR: 'Count::x' is not accessible
23 }
24
25 int main()
26 {
27     Count counter;
28
29     cannotSetX( counter, 3 ); // cannotSetX is not a friend
30     return 0;
31 }

```

Outline

(Previous program without friend declared)

cannotSetX is not a friend of class Count. It cannot access private data.

cannotSetX tries to modify a private variable...

```

Compiling...
Fig07_06.cpp
D:\books\2000\cpphtp3\examples\Ch07\Fig07_06\Fig07_06.cpp(22) :
error C2248: 'x' : cannot access private member declared in
class 'Count'
D:\books\2000\cpphtp3\examples\Ch07\Fig07_06\
Fig07_06.cpp(15) : see declaration of 'x'
Error executing cl.exe.

test.exe - 1 error(s), 0 warning(s)

```

5

[Outline](#)

Program Output

Expected compiler error - cannot
access **private** data

© 2000 Prentice Hall, Inc. All rights reserved.

5

```

1 // Fig. 7.5: fig07_05.cpp
2 // Friends can access private members of a class.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // Modified Count class
9 class Count {
10     friend void setX( Count &, int ); // friend declaration
11 public:
12     Count() { x = 0; } // constructor
13     void print() const { cout << x << endl; } // output
14 private:
15     int x; // data member
16 };
17
18 // Can modify private data of Count because
19 // setX is declared as a friend function of
20 void setX( Count &c, int val )
21 {
22     c.x = val; // legal: setX is a friend of Count
23 }
24
25 int main()
26 {
27     Count counter;
28
29     cout << "counter.x after instantiation: ";
30     counter.print();

```

6

[Outline](#)

1. Class definition

1.1 Declare function a friend

1.2 Function definition

1.3 Initialize Count object

setX a friend of class
Count (can access
private data).

setX is defined normally
and is not a member
function of Count.

Changing **private** variables allowed.

6

```
31 cout << "counter.x after call to setX friend function: ";
32 setX( counter, 8 ); // set x with a friend
33 counter.print();
34 return 0;
35 }
```

7

Outline

- 2. Modify object
- 3. Print results

Program Output

```
counter.x after instantiation: 0
counter.x after call to setX friend function: 8
```

private data was changed.

© 2000 Prentice Hall, Inc. All rights reserved.

7

8

Friend function

- a function of global scope may be declared as friend of both classes
- a member function of one class may be declared as friend of another one.

© 2000 Prentice Hall, Inc. All rights reserved.

8

```

#include <iostream>
using namespace std;

class Foo; // Forward declaration of class Foo in order
           // for example to compile.
class Bar {
private:
    int a;
public:
    Bar(): a(0) {}
    void show(Bar& x, Foo& y) {
        friend void show(Bar& x, Foo& y); // declaration of
        // global friend
    };

    class Foo {
    private:
        int b;
    public:
        Foo(): b(6) {}
        friend void show(Bar& x, Foo& y); // declaration
        // of global friend
        friend void Bar::show(Bar& x, Foo& y); //
        // declaration of friend from other class
    };

    // Definition of a member function of Bar; this member is
    // a friend of Foo
    void Bar::show(Bar& x, Foo& y) {
        cout << "Show via function member of Bar" << endl;
        cout << "Bar::a = " << x.a << endl;
        cout << "Foo::b = " << y.b << endl;
    }

    // Friend for Bar and Foo, definition of global function
    void show(Bar& x, Foo& y) {
        cout << "Show via global function" << endl;
        cout << "Bar::a = " << x.a << endl;
        cout << "Foo::b = " << y.b << endl;
    }

int main() {
    Bar a;
    Foo b;

    show(a,b);
    a.show(a,b);
}

```



THIS POINTER



7.5 Using the **this** Pointer

- **this** pointer
 - Allows objects to access their own address
 - Not part of the object itself
- Examples using **this**
 - For a member function print data member **x**, either

`this->x`

or

`( *this ).x`



```

1 // Fig. 7.7: fig07_07.cpp
2 // Using the this pointer to refer to object members.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 class Test {
9 public:
10     Test( int = 0 );           // default constructor
11     void print() const;
12 private:
13     int x;
14 };
15
16 Test::Test( int a ) { x = a; } // constructor
17
18 void Test::print() const      // ( ) around *this
19 {
20     cout << "      x = " << x
21         << "\n this->x = " << this->x
22         << "\n(*this).x = " << ( *this ).x << endl;
23 }
24
25 int main()
26 {
27     Test testObject( 12 );
28
29     testObject.print();
30
31     return 0;
32 }

```

Outline

- 1. Class definition
- 1.1 Function definition
- 1.2 Initialize object
- 2. Function call

13

Printing **x** directly.

Print **x** using the arrow **->** operator off the **this** pointer.

Printing **x** using the dot **(.)** operator. Parenthesis required because dot operator has higher precedence than *****. Without, interpreted incorrectly as ***(this.x)**.

13

```

      x = 12
this->x = 12
(*this).x = 12

```

Outline

Program Output

14

All three methods have the same result.

14

NEW AND DELETE FUNCTIONS



7.6 Dynamic Memory Allocation with Operators new and delete

- **new** and **delete**
 - Used for dynamic memory allocation
 - **new**
 - Creates an object of the proper size, calls its constructor and returns a pointer of the correct type
 - **delete**
 - Destroys object and frees space
 - Examples of **new**

```
TypeName *typeNamePtr;
```

 - Creates pointer to a **TypeName** object

```
typeNamePtr = new TypeName;
```
 - **new** creates **TypeName** object, returns pointer (which **typeNamePtr** is set equal to)



7.6 Dynamic Memory Allocation with Operators new and delete

– Examples of **delete**

```
delete typeNamePtr;
```

- Calls destructor for **TypeName** object and frees memory

```
Delete [] arrayPtr;
```

- Used to dynamically delete an array

• Initializing objects

```
double *thingPtr = new double( 3.14159 );
```

- Initializes object of type **double** to **3.14159**

```
int *arrayPtr = new int[ 10 ];
```

- Creates a ten element **int** array and assigns it to **arrayPtr**

